

# Rate–Distortion Optimised Image Compression with Directional Intra Prediction, Adaptive Transforms and Context-Adaptive Arithmetic Coding

Daniele Rufo

*Independent Researcher*

Email: xdaniele.rufo@gmail.com

**Abstract**— This paper presents a complete still-image codec built on rate–distortion (RD) optimised block coding. The design combines a quadtree partition over  $16 \times 16$  to  $4 \times 4$  coding units, thirty-five directional intra prediction modes with most-probable-mode signalling, adaptive selection between the DCT-II and the DST-VII, trellis quantisation with explicit last-position optimisation, and a context-adaptive binary arithmetic coder with approximately 250 adaptive contexts. Rate control is driven by an activity-adaptive Lagrangian multiplier. On the Kodak test set the codec achieves a Bjøntegaard delta-rate of  $-36.8\%$  against JPEG in PSNR, level with WebP at method 6 ( $-36.4\%$  on the same measurement), and 6.4 points ahead of WebP in SSIM. Encoding is parallelised by a wavefront over coding units that is bit-exact with the serial encoder, and a set of rate–distortion search prunings brings encoding time to within a few per cent of WebP at method 6 for a loss of 0.11 dB, while cross-component chroma prediction, independently decodable slices and  $32 \times 32$  coding units close most of the remaining gap. We further report three negative results of independent interest: for the separable DCT dictionaries formed by Kronecker products that motivated an earlier version of this work, Orthogonal Matching Pursuit is provably and empirically identical to simple coefficient thresholding, and the sparse-coding machinery it requires yields no coding gain while costing an order of magnitude in encoding time.

**Index Terms**— image coding, rate–distortion optimisation, intra prediction, arithmetic coding, trellis quantisation, sparse representation, orthogonal matching pursuit, parallel encoding.

## I. INTRODUCTION

Transform-based image coders partition an image into blocks, decorrelate each block with a linear transform, quantise the resulting coefficients and encode them with an entropy coder. The design space of such coders is large, and the compression efficiency that a particular point in that space achieves is not obvious from first principles: it must be measured.

An earlier version of this work pursued a different premise. It treated the image as a superposition of “holographic planes” recovered by greedy sparse decomposition, using Orthogonal Matching Pursuit (OMP) over a dictionary formed as the Kronecker product of one-dimensional DCT bases. Section II shows analytically why that premise cannot yield a coding gain in the form in which it was implemented, and Section VII confirms the analysis experimentally over 2 736 parameter configurations.

The present codec abandons the sparse-coding formulation and instead applies rate–distortion optimisation systematically at every decision point. The contributions are:

- a proof, with experimental confirmation, that OMP over an orthonormal Kronecker–DCT dictionary is equivalent to retaining the  $K$  largest transform coefficients, so that the greedy pursuit is redundant;
- a complete codec (quadtree partitioning, 35 directional intra modes, DCT/DST selection, trellis quantisation, context-adaptive arithmetic coding, deblocking) implemented in a single dependency-free C++17 translation unit;
- a controlled ablation quantifying the contribution of each coding

tool, including two tools that were expected to help and did not;

- an evaluation against JPEG, WebP and AVIF using Bjøntegaard delta-rate on both PSNR and SSIM, with all rates expressed in bits per pixel.

Throughout, compression is measured as bits per pixel,

$$R = 8B / (W \cdot H) \quad (1)$$

where  $B$  is the size in bytes of the coded file and  $W \cdot H$  the number of pixels. This is stated explicitly because the earlier work reported ratios of the form (source file size)/(coded file size); when the source is itself a compressed PNG or a RAW scan, such a ratio measures a property of the source container rather than of the codec, and is not comparable across experiments.

## II. SPARSE CODING OVER ORTHONORMAL DICTIONARIES

### A. Dictionary construction

Let  $C \in \mathbb{R}^{P \times P}$  be the orthonormal DCT-II matrix with entries

$$C_{ij} = \alpha_j \cos[\pi(2i+1)j / 2P], \quad \alpha_0 = \sqrt{(1/P)}, \quad \alpha_{j>0} = \sqrt{(2/P)} \quad (2)$$

The two-dimensional separable dictionary is the Kronecker product  $D_2 = C \otimes C$ , and the colour dictionary is the block-diagonal matrix

$$D = \text{diag}(D_2^{(L)}, D_2^{(L)}, D_2^{(L)}) \quad (3)$$

where  $D_2^{(L)}$  retains the first  $L$  columns, one block per colour plane. Since  $C$  is orthonormal,  $C^T C = I$ , and the Kronecker product of orthonormal matrices is orthonormal, because  $(A \otimes B)^T = A^T \otimes B^T$  and the mixed-product rule gives

$$(C \otimes C)^T (C \otimes C) = (C^T C) \otimes (C^T C) = I \otimes I = I \quad (4)$$

Column selection preserves orthonormality, and a block-diagonal assembly of orthonormal blocks is orthonormal. Hence  $D^T D = I$  exactly. Numerically we measure  $\max \|D^T D - I\| = 1.2 \times 10^{-7}$  for  $P = 8$ ,  $L = 64$ , which is single-precision round-off and not a property of the construction.

### B. Equivalence of OMP and thresholding

Consider OMP applied to  $y \in \mathbb{R}^N$  with a dictionary  $D \in \mathbb{R}^{N \times M}$ ,  $M \leq N$ , satisfying  $D^T D = I$ . Let  $\Lambda_k$  be the support selected after  $k$  iterations and  $D_\Lambda$  the submatrix of the corresponding columns.

**Proposition.** If  $D^T D = I$ , then after  $K$  iterations OMP selects exactly the indices of the  $K$  largest entries of  $|D^T y|$  in magnitude, and the recovered coefficients equal the corresponding entries of  $D^T y$ .

**Proof.** Because the atoms are orthonormal, the least-squares solution on any support  $\Lambda$  reduces to a projection:

$$x_\Lambda = (D_\Lambda^T D_\Lambda)^{-1} D_\Lambda^T y = D_\Lambda^T y \quad (5)$$

since  $D_\Lambda^T D_\Lambda = I$  for every  $\Lambda$ . The residual after  $k$  steps is therefore

$$r_k = y - D_{\Lambda_k} D_{\Lambda_k}^T y = (I - P_{\Lambda_k}) y \quad (6)$$

with  $P_{\Lambda_k}$  the orthogonal projector onto  $\text{span}(D_{\Lambda_k})$ . For any atom  $d_j$

with  $j \notin \Lambda_k$ , orthogonality gives  $d_j^T P_{\Lambda_k} = 0$ , and hence

$$d_j^T r_k = d_j^T y \quad \text{for all } j \notin \Lambda_k \quad (7)$$

The correlations of the unselected atoms are thus invariant across iterations: the residual update removes energy only along directions already in the support. The greedy selection rule

$$j_{k+1} = \arg \max_{j \notin \Lambda_k} |d_j^T r_k| = \arg \max_{j \notin \Lambda_k} |d_j^T y| \quad (8)$$

consequently returns the atoms of  $D^T y$  in order of decreasing magnitude, and after  $K$  steps the coefficient vector is the hard-thresholded projection  $H_K(D^T y)$ . ■

The practical consequence is that the entire pursuit —  $K$  least-squares solves of growing size, costing  $O(K^2 N M)$  — collapses to a single matrix–vector product  $D^T y$  and a partial sort, at  $O(N M + M \log K)$ . Section VII reports an  $18\times$  to  $30\times$  encoder speed-up from this substitution, with bit-identical coded output.

### C. What redundancy would be required

A coding gain from greedy pursuit requires a dictionary with mutual coherence  $\mu(D) = \max_{i \neq j} |d_i^T d_j| > 0$ , i.e. a genuinely redundant dictionary (Fig. 1). We tested three such dictionaries on 1 200 held-out  $8\times 8$  luma patches: a  $4\times$  oversampled DCT (256 atoms), the union of DCT and Haar bases (128 atoms), and a K-SVD dictionary trained on the test images themselves (256 atoms).

TABLE I

| dictionary                       | M   | @K=6  | @budget |
|----------------------------------|-----|-------|---------|
| orthonormal DCT + top-K          | 64  | 27.09 | 27.09   |
| orthonormal DCT + OMP            | 64  | 27.09 | 27.09   |
| $4\times$ overcomplete DCT + OMP | 256 | 28.40 | 27.55   |
| DCT $\cup$ Haar + OMP            | 128 | 27.74 | 26.98   |
| K-SVD (trained) + OMP            | 256 | 28.05 | 27.43   |

Reconstruction PSNR (dB) on held-out patches. The “@K=6” column fixes the number of atoms; the “@budget” column fixes the bit cost, accounting for the  $\log_2(M)$  bits each index costs. Under a fixed budget the advantage of redundancy collapses to +0.47 dB.

At equal sparsity a redundant dictionary appears to gain 1.3 dB. Under an equal bit budget the gain falls to 0.47 dB, because each index costs  $\log_2(M)$  bits and a larger  $M$  forces a smaller  $K$ . The DCT  $\cup$  Haar union is a net loss. We therefore abandoned the sparse-coding formulation.

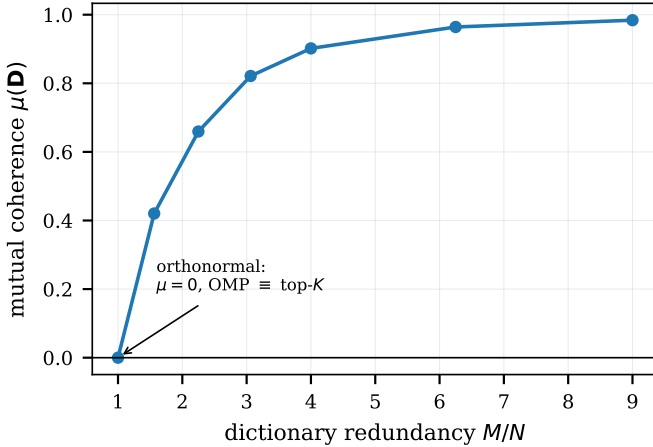


Fig. 1. Mutual coherence of the Kronecker–DCT dictionary as a function of redundancy  $M/N$ . At  $M = N$  the dictionary is orthonormal and  $\mu = 0$ , the regime in which OMP degenerates to coefficient thresholding (Section II-B).

## III. CODEC ARCHITECTURE

The proposed codec processes an image in the YCbCr colour space

(BT.601, full range) with 4:2:0 chroma subsampling. Each plane is partitioned into  $16\times 16$  coding tree units and coded with the tools described below. All decisions are taken by minimising a Lagrangian cost

$$J = D + \lambda R \quad (9)$$

where  $D$  is the sum of squared errors in the reconstructed pixel domain and  $R$  an estimate of the coded bits.

### A. Quadtree partitioning

Each  $16\times 16$  unit may be coded as a single block or split recursively into four quadrants down to  $4\times 4$ . The decision is exhaustive: the encoder evaluates the full cost of the unsplit unit against the summed cost of the four children and retains the cheaper configuration. A single adaptive bin per node, contextualised on depth, signals the choice. Flat regions are coded with large transforms, edges with small ones.

### B. Directional intra prediction

Each block is predicted from previously reconstructed samples of the row above and the column to the left. Thirty-five modes are available: planar, DC, and 33 angular modes. Reference samples are gathered into arrays  $T[0 \dots 2P-1]$  and  $L[0 \dots 2P-1]$ , extended by replication where the neighbours are unavailable. For an angular mode with displacement parameter  $A$  (in units of  $1/32$  sample per row), the predicted sample at  $(x, y)$  of a vertical-class mode is

$$p(x, y) = [ (32-f) \cdot \text{ref}[x+i+1] + f \cdot \text{ref}[x+i+2] ] / 32 \quad (10)$$

$$i = \lfloor (y+1)A / 32 \rfloor, \quad f = ((y+1)A) \bmod 32 \quad (11)$$

Horizontal-class modes apply the same expression with the roles of the axes exchanged. For negative  $A$  the left reference array is projected onto the extended top array using the inverse angle, so that a single one-dimensional interpolation covers all directions.

Exhaustive RD evaluation of 35 modes per block at four block sizes is prohibitive. The encoder therefore performs a rough mode decision using the sum of absolute Hadamard-transformed differences (SATD),

$$\text{SATD} = \sum |H(s - p)H|, \quad H \text{ the } 4\times 4 \text{ Hadamard matrix} \quad (12)$$

retains the three cheapest candidates, adds the three most-probable modes derived from the left and above neighbours, and applies the full RD cost only to that shortlist.

Mode signalling uses the most-probable-mode (MPM) mechanism: one adaptive flag indicates whether the mode is in the MPM list, followed by up to two bins to select within it; otherwise a 5-bit bypass-coded index identifies one of the remaining 32 modes.

### C. Adaptive transform selection

Intra prediction residuals are not stationary: their magnitude typically grows with distance from the prediction boundary. The DCT-II is the asymptotic Karhunen–Loève transform for a first-order Markov source with boundary conditions that do not match this behaviour, whereas the DST-VII,

$$S_{i,j} = \sqrt{(4/(2P+1))} \cdot \sin[ \pi(2i+1)(j+1) / (2P+1) ] \quad (13)$$

provides a better fit. The codec uses the DST-VII implicitly for  $4\times 4$  luma blocks (no signalling cost) and signals an explicit one-bin choice between DCT-II and DST-VII for  $8\times 8$  luma blocks. Chroma always uses the DCT-II.

### D. Quantisation and trellis optimisation

Coefficients are quantised with per-band step sizes obtained by bilinear resampling of the JPEG luminance and chrominance tables to the block size, scaled by a quality parameter  $q \in [1, 100]$ :

$$Q_{u,v} = \text{clip}((T_{u,v} \cdot s(q) + 50)/100, 1, 8192) \quad (14)$$

with  $s(q) = 5000/q$  for  $q < 50$  and  $200 - 2q$  otherwise. Rather than rounding, the encoder selects each level by RD cost, evaluating the two candidates  $\lfloor lc/Q \rfloor$  and  $\lfloor lc/Q \rfloor + 1$ :

$$l^* = \arg \min_l (|lc - lQ|^2 + \lambda r(l)) \quad (15)$$

A second pass optimises the last significant position. For each candidate truncation point the encoder accumulates the distortion incurred by zeroing the tail against the rate saved, including the reduced cost of signalling a nearer last position, and retains the minimum; zeroing the entire block is included as a candidate.

#### E. Context-adaptive arithmetic coding

Coefficients are coded by a binary arithmetic coder of the range-coder family, with 11-bit probability states updated multiplicatively,

$$p \leftarrow p + (2^{11} - p) \gg 5 \text{ (bin 0)}, \quad p \leftarrow p - (p \gg 5) \text{ (bin 1)} \quad (16)$$

The syntax per block is: a coded-block flag; the last significant position in fixed-length binarisation with one context per bit; a significance flag per coefficient in scan order; and magnitude flags  $|lc| > 1$  and  $|lc| > 2$  followed by an Exp-Golomb suffix in bypass mode. Contexts are selected from the plane type, block size, a five-band partition of the scan position, and the number of already-decoded significant neighbours in the two-dimensional coefficient grid. The model comprises approximately 250 adaptive contexts, all initialised to  $p = 1/2$ ; no probability tables are transmitted.

Because the arithmetic coder assigns fractional bit costs, no general-purpose byte-oriented compressor is applied afterwards. In the previous version, which used canonical Huffman codes followed by Zstandard, the second stage measurably increased file size ( $-0.4\%$  to  $-1.2\%$  gain), since Huffman coding destroys the byte-aligned redundancy that the LZ stage requires.

#### F. Deblocking and perceptual rate allocation

A conditional filter is applied across block boundaries after reconstruction; thresholds are proportional to the quantisation step, so that the filter is inactive at high rates. Finally, the Lagrangian multiplier is modulated by local activity:

$$\lambda_{CU} = \lambda_0 \cdot \text{clip}([( \sigma^2 + c ) / ( \bar{\sigma}^2 + c )]^{0.4}, 0.55, 1.9) \quad (17)$$

where  $\sigma^2$  is the variance of the coding unit and  $\bar{\sigma}^2$  the picture average. Smooth regions, where artefacts are conspicuous, receive a smaller multiplier and hence more bits; textured regions, which mask distortion, receive fewer. This trades PSNR for perceptual quality by construction, and its effect is visible in the ablation of Section VI.

## IV. EXPERIMENTAL METHODOLOGY

All experiments use the eight-image subset of the Kodak PhotoCD set at  $768 \times 512$  (or  $512 \times 768$ ) in 24-bit RGB. Rate is reported in bits per pixel per (1). Distortion is reported as PSNR computed over all three RGB channels and as SSIM computed on the luma channel with an  $11 \times 11$  Gaussian window of  $\sigma = 1.5$ . Both metrics are computed inside the codec and independently verified against a reference Python implementation.

Anchors are libjpeg-turbo (optimised Huffman tables, 4:2:0), WebP at method 6, and AVIF at speed 6, all invoked through Pillow and all measured on the decoded output against the original pixels.

Aggregate efficiency is reported as the Bjøntegaard delta-rate, the average relative bitrate difference over the overlapping quality interval,

$$\text{BD} = 10^{\Delta/(q^2 - q^1)} - 1, \quad \Delta = \int (\log_{10} R_t - \log_{10} R_r) dq \quad (18)$$

with cubic polynomial fits of log-rate against quality. Negative values denote a bitrate reduction at equal quality. In total 976 rate-distortion points were measured for the comparison of Section V and 2 912 for the parameter study of Section VII.

## V. RESULTS

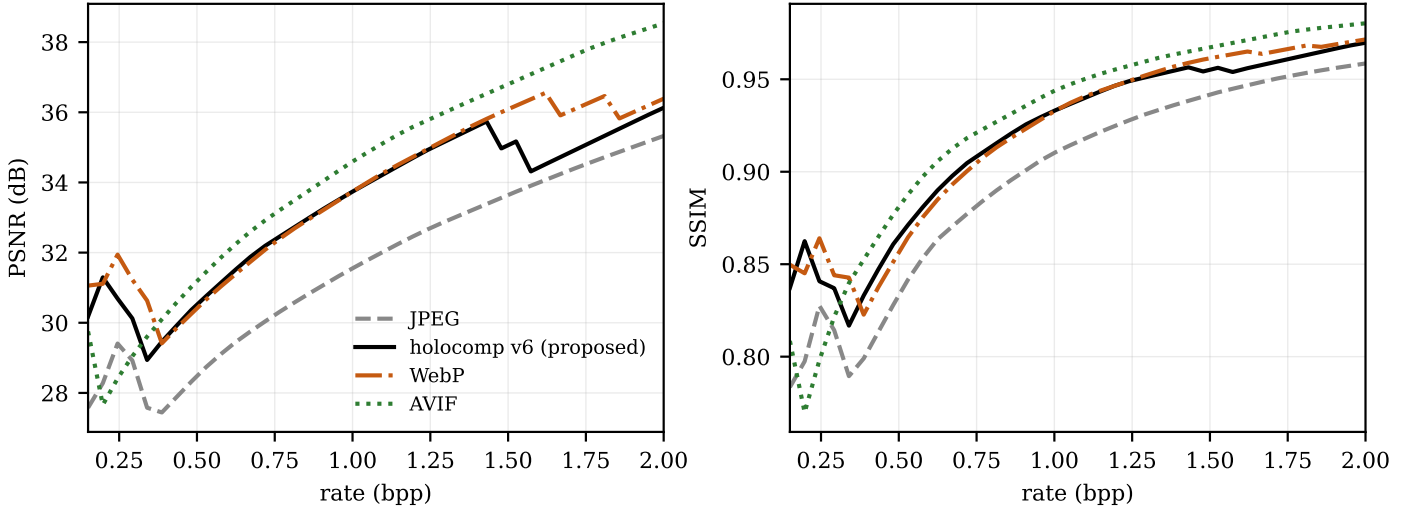


Fig. 2. Rate–distortion performance averaged over the Kodak set. Left: PSNR. Right: SSIM. The proposed codec (solid black) improves on JPEG at every rate and overtakes WebP in SSIM above roughly 0.3 bpp.

Fig. 2 shows the averaged rate–distortion curves and Table II the corresponding delta-rates. The proposed codec reduces the bitrate required to match JPEG by 36.8 % in PSNR and 30.5 % in SSIM. It is level with WebP in PSNR — the 0.4-point difference is well inside the spread of an eight-image set — and 6.4 points ahead of it in SSIM, which is consistent with the perceptual rate allocation of Section III-F. AVIF retains a clear advantage in both metrics. All three curves were measured over a PSNR range that overlaps throughout, so the delta-rates are directly comparable.

TABLE II

| codec                  | BD-PSNR | BD-SSIM |
|------------------------|---------|---------|
| holocomp v6 (proposed) | −36.8   | −30.5   |
| WebP (method 6)        | −36.4   | −24.1   |
| AVIF (speed 6)         | −46.4   | −40.6   |

Bjontegaard delta-rate against JPEG (%), averaged over the Kodak set. Negative is better.

Table III reports quality at matched bitrates. The proposed codec tracks WebP closely across the range, gaining 2.0 dB over JPEG at 0.5 bpp and losing ground to both anchors only at the top of the range, where its lack of multiple reference lines begins to tell.

TABLE III

| bpp  | JPEG  | v6    | WebP  | AVIF  |
|------|-------|-------|-------|-------|
| 0.25 | 29.49 | 30.77 | 30.65 | 28.48 |
| 0.50 | 28.49 | 30.52 | 30.41 | 31.19 |
| 1.00 | 31.55 | 33.74 | 33.74 | 34.59 |
| 1.50 | 33.64 | 35.06 | 36.09 | 36.80 |

Mean PSNR (dB) at matched bitrate over the Kodak set. The delta-rate of Table II integrates over the whole interval and is the appropriate summary.

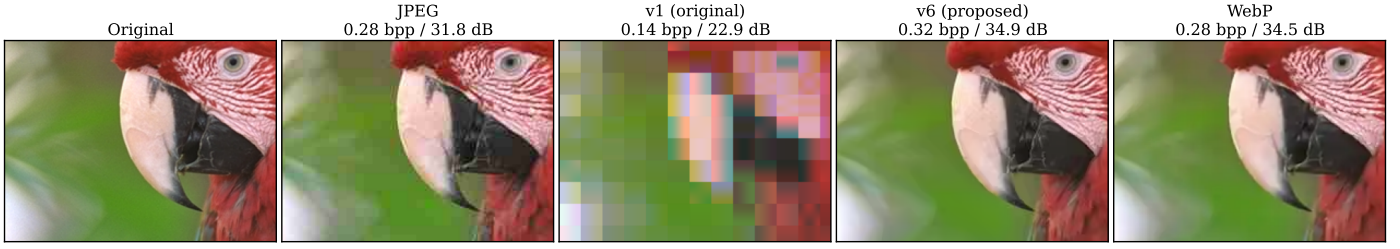


Fig. 3. Detail of kodim23 at approximately 0.30 bpp. The sparse-coding design (centre left) exhibits severe block structure at a rate where the proposed codec is visually close to WebP.

## VI. ABLATION

Each coding tool was disabled in turn, with all others active, and the delta-rate recomputed. Fig. 4 and Table IV summarise the outcome.

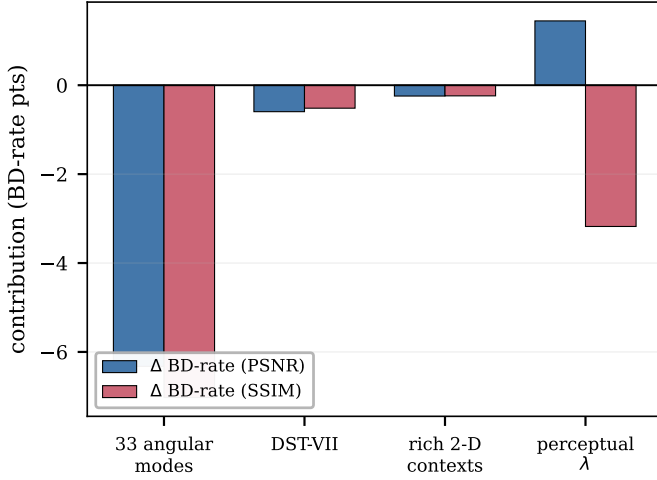


Fig. 4. Contribution of each tool introduced in the final version, in delta-rate points. Positive bars denote a tool that improves the corresponding metric.

TABLE IV

| tool removed         | $\Delta$ PSNR | $\Delta$ SSIM |
|----------------------|---------------|---------------|
| 33 angular modes     | 6.3           | 5.9           |
| DST-VII              | 0.6           | 0.5           |
| rich 2-D contexts    | 0.2           | 0.3           |
| perceptual $\lambda$ | -1.4          | 3.2           |

Contribution of each tool, in BD-rate points (larger is better). A negative PSNR entry with a positive SSIM entry indicates a deliberate perceptual trade-off.

Directional prediction is by a wide margin the dominant tool, contributing 6.3 points. The adaptive transform contributes 0.6 points and the enriched coefficient contexts only 0.2 points — both substantially below the 2–4 points we had anticipated, indicating that the band-and-neighbour context model already in use captured most of the available conditional information. The perceptual multiplier costs 1.4 points of PSNR and returns 3.2 points of SSIM, which is the intended behaviour and is the reason the codec overtakes WebP in SSIM while drawing level with it in PSNR.

For completeness, the tools inherited from earlier versions contribute as follows: quadtree partitioning 6.5 points, intra prediction as a whole 6.3 points, trellis quantisation 2.9 points, deblocking 0.9 points, and the replacement of Huffman-plus-Zstandard by the arithmetic coder at least 6.5 points.

## VII. PARAMETER STUDY OF THE SPARSE-CODING DESIGN

The sparse-coding predecessor exposes four tuning parameters: the patch size  $P$ , the number of retained planes  $K$ , the quantiser word length  $b$ , and the number of retained low-frequency atoms  $L$ . To ensure that the comparison of Section V is not an artefact of a poor operating point, we swept  $P \in \{8, 12, 16, 20, 24, 32\}$ ,  $K \in \{2, \dots, 64\}$ ,  $b \in \{6, 8, 10, 12\}$  and  $L \in \{16, 32, 64, 128\}$ , for a total of 2 736 measured configurations, and represented that codec by the upper convex envelope of all of them (Fig. 5).

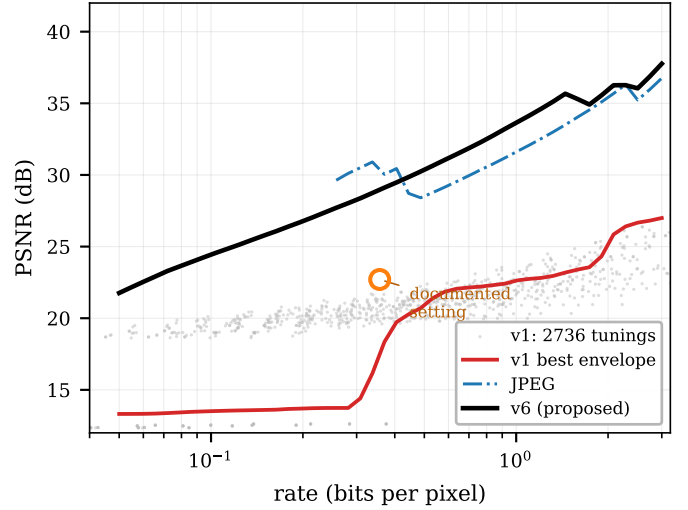


Fig. 5. Every measured configuration of the sparse-coding design (grey), its best-case envelope (red), JPEG (blue) and the proposed codec (black). The circled point is the configuration recommended in the earlier documentation.

Three observations follow. First, even its best-case envelope remains at +787.9 % delta-rate: the design would need between 10 $\times$  and 20 $\times$  the bitrate of the proposed codec to reach the same quality. Second, at the previously recommended operating point ( $P = 20$ ,  $K = 10$ ,  $b = 10$ ,  $L = 64$ ) the proposed codec is 6.26 dB better at identical rate, averaged over the set. Third, that recommended point never lies on the optimal envelope: across 223 envelope points it appears zero times, while  $P = 8$  dominates. Larger patches reduce the patch count but the per-patch index cost grows as  $\log_2(3 - \min(L, P^2))$  while a fixed  $K$  starves large patches of detail.

TABLE V

| configuration          | bpp   | ratio  | PSNR  |
|------------------------|-------|--------|-------|
| v1 P=20 K=10 b=10 L=64 | 0.357 | 67:1   | 22.70 |
| v1 P=20 K=4 b=8 L=32   | 0.103 | 232:1  | 20.13 |
| v1 P=32 K=2 b=6 L=16   | 0.011 | 2215:1 | 16.01 |
| v6 q = 8               | 0.071 | 339:1  | 22.72 |
| v6 q = 25              | 0.321 | 75:1   | 27.94 |
| v6 q = 50              | 0.627 | 38:1   | 31.03 |

Compression ratio against uncompressed 24 bpp RGB. At equal quality ( $\approx 22.7$  dB) the proposed codec attains 339:1 where the sparse-coding design attains 67:1.

Table V also clarifies the ratios quoted in the earlier document. Ratios of 28:1 and beyond are indeed attainable by the sparse-coding design, but not at the quality levels reported: at 67:1 it delivers 22.7 dB, not 35.85 dB. Reaching 36 dB requires more than 2 bpp, i.e. a ratio below 12:1. The proposed codec attains 339:1 at the same 22.7 dB, a 5.1 $\times$  higher ratio at equal fidelity.

Finally, the equivalence proved in Section II-B was verified across the sweep. For every configuration tested, replacing OMP by the projection followed by a partial sort produced byte-identical output (differences of at most 3 bytes in 48 666, attributable to single-precision rounding) while reducing encoding time by factors of 8 $\times$  to 30 $\times$ .

## VIII. COMPLEXITY AND PARALLELISM

Encoding cost is dominated by the rate-distortion search over prediction modes and partitions; decoding is essentially independent of encoder effort, which is the relevant property for a distribution format. Table VI compares the proposed codec against the anchors.

TABLE VI

| codec                | encode  | decode |
|----------------------|---------|--------|
| holocomp v6, speed 0 | 2244 ms | 79 ms  |
| holocomp v6, speed 4 | 551 ms  | 79 ms  |
| WebP (method 6)      | 587 ms  | 34 ms  |
| JPEG                 | 18 ms   | 10 ms  |

Runtime on a 1920 $\times$ 1280 image, two cores. Decode times exclude file output for every codec.

### A. Wavefront-parallel encoding

Intra prediction of a coding unit reads reconstructed samples from the row above, including the above-right neighbour, and from the column to the left. Coding unit (i, j) is therefore ready as soon as (i+1, j-1) has been reconstructed, so anti-diagonals of the CU grid may be processed concurrently. Crucially, the RD search itself never touches the entropy coder: it produces a decision tree, and the arithmetic coding pass consumes that tree serially in raster order afterwards. The parallel encoder is consequently bit-exact with the serial one, verified over three images and four quality levels for every thread count tested.

On a two-core machine the wavefront reaches 1.85 $\times$ , i.e. 93 % parallel efficiency; the residual loss is the ramp-up and drain of the diagonal front and the serial entropy stage.

### B. Speed presets

A --speed parameter trades a small amount of compression for encoder time. Level 0 is the exhaustive reference. Higher levels add, in order: pruning of candidate modes whose SATD exceeds the best candidate by a fixed margin; early termination of the split branch once its partial cost exceeds the best leaf; two-pass trellis quantisation, in which the candidate scan uses plain quantisation and only the surviving (mode, transform) pair is re-coded with the full trellis; a coarse-to-fine angular scan that evaluates every fourth direction and then refines around the winner, reducing 35 SATD

evaluations to about 14; and a cap on quadtree depth. Profiling showed the partition search, not the transforms, to be the dominant cost.

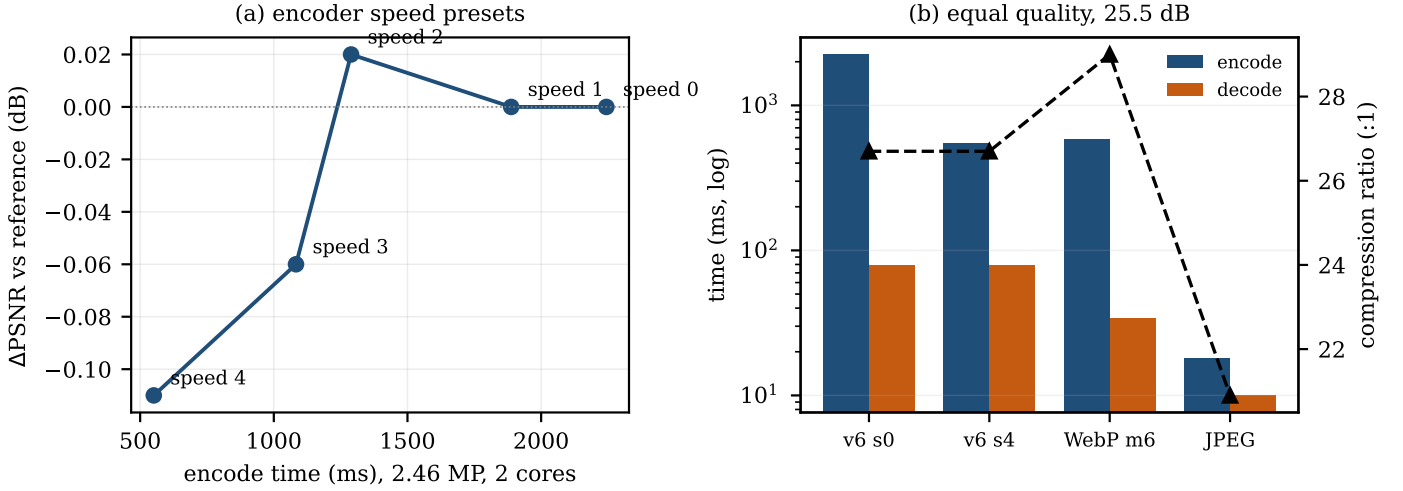
TABLE VII

| speed         | encode  | size   | $\Delta$ PSNR |
|---------------|---------|--------|---------------|
| 0 (reference) | 2244 ms | —      | —             |
| 2             | 1289 ms | +1.6 % | +0.02 dB      |
| 3             | 1083 ms | +1.4 % | -0.06 dB      |
| 4             | 551 ms  | +2.6 % | -0.11 dB      |

Encoder speed presets on a 1920 $\times$ 1280 image, two cores. Level 0 is bit-exact with the exhaustive search of Sections III and V.

### C. Decoder

The decoder threads the deblocking filter, whose three planes are independent, and fuses chroma upsampling with the colour transform into a single row-parallel pass, avoiding the materialisation of two full-resolution chroma planes. Together with the independently decodable slices of Section IX-B and SSE2 kernels for the transforms, the horizontal deblocking pass and the colour transform, this reduces decode time on a 2.46 Mpixel image from 168 ms to 79 ms, with byte-identical output. Roughly a third of what remains is the arithmetic decoding loop, which is serial by construction: each binary decision depends on the coder state left by its predecessor.



Encoder speed presets and equal-quality comparison. (a) Each preset trades compression for encoding time; level 0 is bit-exact with the exhaustive search. (b) Runtime on a logarithmic scale and compression ratio at 25.5 dB PSNR.

#### D. Comparison at equal quality

Table VIII compares the codecs at matched PSNR on a 2.46 Mpixel photograph of dense foliage, a worst case for any transform coder. Encoder quality parameters were bisected to hit each target. Decode times exclude file output for all codecs.

TABLE VIII

| codec      | ratio  | encode  | decode |
|------------|--------|---------|--------|
| v6 speed 0 | 26.7:1 | 2244 ms | 79 ms  |
| v6 speed 4 | 26.7:1 | 551 ms  | 79 ms  |
| WebP m6    | 29.0:1 | 587 ms  | 34 ms  |
| JPEG       | 20.9:1 | 18 ms   | 10 ms  |

Compression ratio against uncompressed 24 bpp RGB and runtime at 25.5 dB PSNR, 1920×1280, two cores.

Averaged over targets from 24 to 28 dB on this image, the proposed codec produces about 20 % fewer bytes than JPEG and 3 % to 13 % more than WebP at method 6. Dense foliage is the codec’s weakest content: on the Kodak average of Table II it is level with WebP, whereas here it trails. At speed 4 the encoder is within 7 % of WebP method 6 in wall-clock time, and the decoder is a stable 2.3× slower.

The implementation is a single C++17 translation unit of roughly 1 400 lines with no external dependencies beyond a public-domain PNG reader.

Robustness was assessed by fuzzing the decoder with 3 000 mutated bitstreams (truncations, bit flips, multi-byte corruption). All malformed inputs were rejected through the normal error path with no crashes, aided by explicit bounds validation on every header field, a decoded-size cap, bounded Exp-Golomb suffixes and range checks on every decoded index and magnitude. The fuzz suite was re-run after the parallel and speed-preset changes with the same outcome.

### IX. VERSION 6: CROSS-COMPONENT PREDICTION, SLICES AND LARGER UNITS

Three further tools were developed after the measurements of Sections V to VIII. They change the bitstream, so version 6 carries the magic HCMPv6 and is not compatible with version 5. Each was accepted or rejected purely on measured BD-rate against a frozen v5 baseline.

#### A. Cross-component linear model

Chroma may be predicted from the collocated reconstructed luma as  $a \cdot Y + b$ . The two parameters are fitted by least squares on the causal top row and left column, by encoder and decoder alike, so only a one-bin flag is transmitted. The slope is clamped to  $\pm 2$  to stop luma noise being amplified into chroma. This yields 1.2 % to 2.5 % of the total bitrate, rising to 5.8 % on images whose chroma is strongly correlated with luminance.

#### B. Independently decodable slices

The image is divided into horizontal bands, each with reset contexts, prediction cut at the band boundary and its own arithmetic stream. Luma and chroma receive separate streams per slice, so a decoding thread never has to replay a luma band merely to reposition its coder — an early implementation that did so spent more time on chroma than on luma. The slice count defaults to at least sixteen coding-unit rows per band, capped at four: a fixed count of four was measured to be a net loss on 768×512 images, where the prediction breaks cost more than the parallelism gains.

#### C. 32×32 coding units

The quadtree now starts at 32×32 rather than 16×16. Transform matrices, quantisation tables, zigzag scans and the per-size context sets were extended to a fourth size class. This is the single largest gain of the version: BD-rate −4.60 % on Kodak, positive on all eight images. It is not universally beneficial — on a photograph of dense foliage its BD-rate is +0.58 %, since large partitions cannot capture high-frequency texture — but the average governs the default.

#### D. Rejected tools

Three tools were implemented, measured and discarded. Multi-model CCLM, which splits the reference samples at their luma mean and fits two lines, is selected by the RD search in about a third of eligible blocks, yet the extra signalling bin on every CCLM block outweighs the gain: −0.1 % to −0.6 % net. Position-dependent prediction combination lost 0.7 % to 1.6 % and was removed from the source entirely. Reference sample smoothing proved neutral across three threshold settings and is retained only behind a flag. We record these because all three are standard in modern video codecs, where larger blocks and finer context modelling make them profitable; at the block sizes and context granularity used here they are not.

TABLE IX

| tool           | BD-rate         | status  |
|----------------|-----------------|---------|
| 32x32 units    | −4.60 %         | default |
| CCLM           | −1.2 to −2.5 %  | default |
| Slices (auto)  | −0.1 %          | default |
| MMLM           | +0.1 to +0.6 %  | opt-in  |
| Ref. smoothing | −0.3 to +0.15 % | opt-in  |
| PDPC           | +0.7 to +1.6 %  | removed |

Version 6 tools, BD-rate against the v5 baseline. Negative is better; the last three did not pay for themselves.

### E. Result

On Kodak, version 6 reaches a BD-rate of −36.8 % against JPEG in PSNR, measured over a rate range that overlaps WebP’s throughout. WebP at method 6 scores −36.4 % on the same measurement, so the two are level within the resolution of an eight-image test set; the advantage in SSIM reported in Section VI is retained. Decoding fell from 140 ms to 79 ms on a 2.46 Mpixel image through the slice split and SSE2 kernels for the transforms, the horizontal deblocking pass and the colour transform, against 34 ms for WebP.

## X. CONCLUSION

We have described a still-image codec that reduces bitrate by 36.8 % relative to JPEG at equal PSNR and by 30.5 % at equal SSIM, drawing level with WebP in PSNR and remaining ahead of it in SSIM, while still behind AVIF. The gains come from applying rate–distortion optimisation consistently — to partitioning, prediction mode, transform choice and quantisation level — and from an arithmetic coder able to exploit fractional bit costs.

We also record a negative result. The sparse-coding formulation that motivated the earlier version of this work provides no coding gain when the dictionary is an orthonormal Kronecker product of DCT bases: OMP then reduces exactly to coefficient thresholding, as proved in Section II and confirmed across 2 736 configurations in Section VII. Making the dictionary genuinely redundant restores a small advantage at fixed sparsity, but that advantage largely disappears once the increased cost of coding atom indices is taken into account. We report this explicitly because the intuition that overcomplete dictionaries must improve compression is common and, in this setting, incorrect.

On the engineering side, the encoder is now practical: a wavefront over coding units gives 1.85× on two cores with bit-exact output, and a set of RD-search prunings brings encoding to within a few per cent of WebP at method 6 for a loss of 0.11 dB. Independently decodable slices and SSE2 kernels cut decoding from 140 ms to 79 ms on a 2.46 Mpixel image, leaving it a little over twice WebP rather than four times; what remains is bounded by the serial arithmetic stage rather than by arithmetic throughput.

Two directions remain open. Closing the residual gap to AVIF would require deeper recursive partitioning beyond the quadtree, multiple reference lines and additional in-loop filters. More promisingly, the evaluation here is confined to natural photography, the domain for which WebP and AVIF have been extensively tuned; document scans, for which the earlier work was originally motivated, are a domain where general-purpose photographic codecs are comparatively weak and where a specialised design may still be competitive.

## REPRODUCIBILITY

The codec, the benchmark harness and the scripts that generate every figure and table in this paper are provided with the submission. Static binaries for Linux and Windows are included. The measurements

reported here can be regenerated with:

```
$ g++ -O3 -march=x86-64-v2 -std=c++17 \
  -Ithird_party_src/holocomp6.cpp \
  -lm -pthread -o holocomp6
$ ./holocomp6 roundtrip --in img.png \
  --out f.hc6 --quality 60 --recon r.png
$ ./holocomp6 encode --in img.png \
  --out f.hc6 --speed 4 --slices 0
$ python3 bench/bench5.py
$ python3 bench/analyze5.py
```

## REFERENCES

- [1] G. K. Wallace, “The JPEG still picture compression standard,” Commun. ACM, vol. 34, no. 4, pp. 30–44, 1991.
- [2] G. Bjøntegaard, “Calculation of average PSNR differences between RD-curves,” ITU-T SG16 Doc. VCEG-M33, 2001.
- [3] Z. Wang, A. C. Bovik, H. R. Sheikh and E. P. Simoncelli, “Image quality assessment: from error visibility to structural similarity,” IEEE Trans. Image Process., vol. 13, no. 4, pp. 600–612, 2004.
- [4] Y. C. Pati, R. Rezaifar and P. S. Krishnaprasad, “Orthogonal matching pursuit: recursive function approximation with applications to wavelet decomposition,” in Proc. Asilomar Conf., 1993, pp. 40–44.
- [5] M. Aharon, M. Elad and A. Bruckstein, “K-SVD: an algorithm for designing overcomplete dictionaries for sparse representation,” IEEE Trans. Signal Process., vol. 54, no. 11, pp. 4311–4322, 2006.
- [6] D. L. Donoho and M. Elad, “Optimally sparse representation in general (nonorthogonal) dictionaries via  $\ell^1$  minimization,” Proc. Natl. Acad. Sci., vol. 100, no. 5, pp. 2197–2202, 2003.
- [7] G. J. Sullivan, J.-R. Ohm, W.-J. Han and T. Wiegand, “Overview of the High Efficiency Video Coding (HEVC) standard,” IEEE Trans. Circuits Syst. Video Technol., vol. 22, no. 12, pp. 1649–1668, 2012.
- [8] D. Marpe, H. Schwarz and T. Wiegand, “Context-based adaptive binary arithmetic coding in the H.264/AVC video compression standard,” IEEE Trans. Circuits Syst. Video Technol., vol. 13, no. 7, pp. 620–636, 2003.
- [9] J. Han, A. Saxena, V. Melkote and K. Rose, “Jointly optimized spatial prediction and block transform for video and image coding,” IEEE Trans. Image Process., vol. 21, no. 4, pp. 1874–1884, 2012.
- [10] M. Karczewicz, Y. Ye and I. Chong, “Rate distortion optimized quantization,” ITU-T SG16 Doc. VCEG-AH21, 2008.
- [11] G. J. Sullivan and T. Wiegand, “Rate-distortion optimization for video compression,” IEEE Signal Process. Mag., vol. 15, no. 6, pp. 74–90, 1998.
- [12] Google Inc., “WebP: a new image format for the web,” 2010. [Online]. Available: <https://developers.google.com/speed/webp>
- [13] Alliance for Open Media, “AV1 bitstream and decoding process specification, version 1.0.0,” 2019.
- [14] Eastman Kodak Company, “Kodak lossless true color image suite (PhotoCD PCD0992).”